

【实验报告】

课程名： 计算机组成与体系结构（H）

实验名称： lab1 算术运算五级流水线 CPU

学生： 杨箫羽

学号： 24300240069

时间： 2026 年 3 月 15 日

一、实验内容与实现思路说明

Lab1 要求利用五级流水线 cpu 架构完成面向 RISC-v 指令集中 12 条算数指令的 cpu 设计，也就是在给定 ISA 的基础上遵循 5 级 pipeline 的微架构进行细节的内核设计。

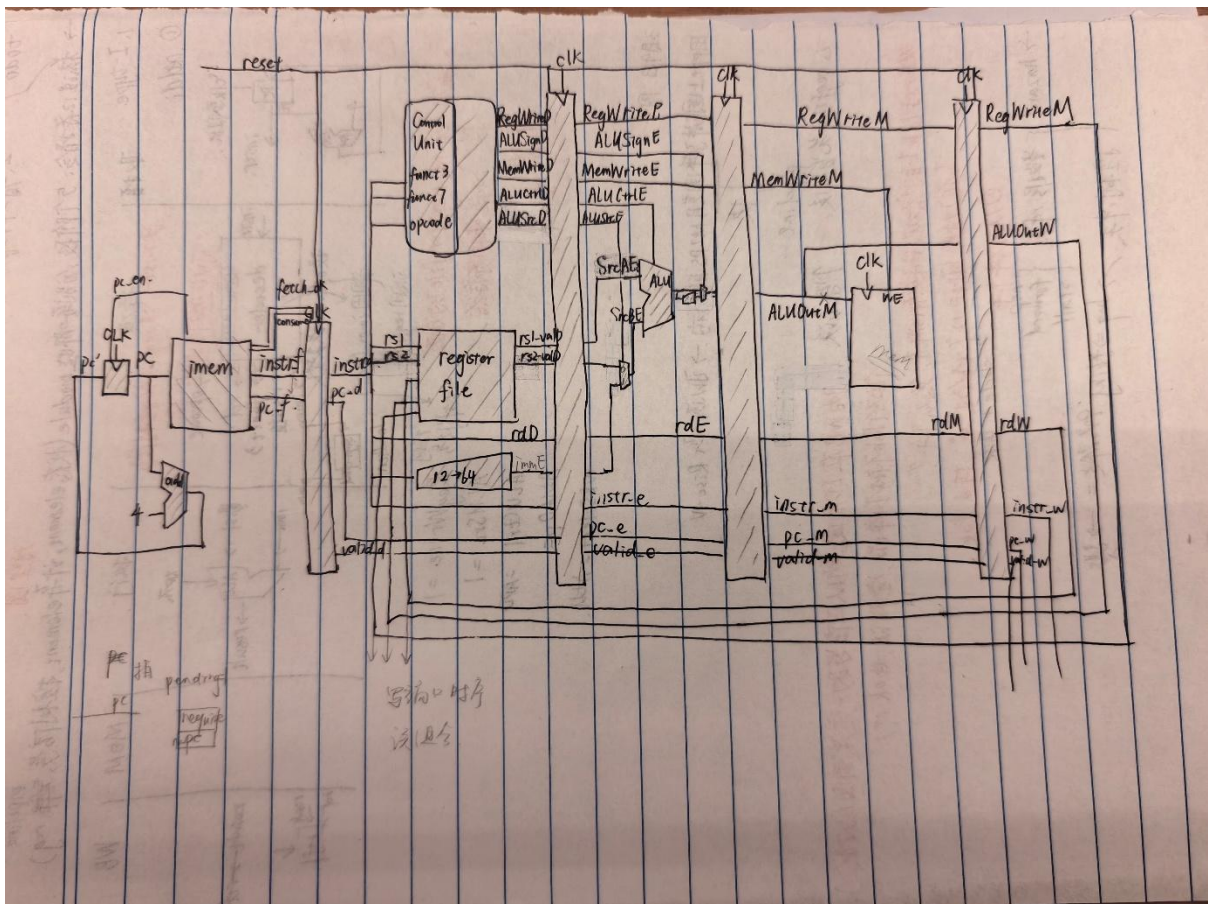
我在了解实验要求并阅读 RISC-v 指令集的细节之后，参考课堂上老师给的 MIPS 流水线 cpu 的图纸，绘制了 RISC-v 的 cpu 图纸，然后完全对照图纸进行功能模块的设计，最后再通过 datapath 模块进行所有功能模块的串联，core 调用 datapath 即可。在代码结构设计的过程中，我尽量将时序逻辑的部分放在功能模块里面，然后 datapath 只进行组合串联（除了最后提交寄存器时在 datapath 中又打了一拍）。

```
root@xiaoyu:~/26-Arch# make test-lab1
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'clint' at [0x38000000, 0x3800ffff]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite' at [0x40600000, 0x4060000c]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite1' at [0x23333000, 0x2333300f]
The first instruction of core 0 has committed. DiffTest enabled.
[src/cpu/cpu-exec.c:393,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x0000000000010004
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 2357 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 16385
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 6951633 instr/s
Program execution has ended. To restart the program, exit NEMU and run again.
Program execution has ended. To restart the program, exit NEMU and run again.
```

下面分三部分给出 CPU 设计的具体说明、调试过程说明和 LLM 使用说明。

二、CPU 设计的具体说明

1、RISC-v 的 cpu 图纸



(除了取指部分，后期调试时将 instr_mem、pc_reg 和 alu_adder 模块合并成 instr_mem 了，其他的功能模块的代码实现，包括接口等，完全按照图纸进行)

2、各个功能模块的说明（按照 5 个阶段的逻辑顺序）

下面具体说明各个模块的功能。

- (1) instr_mem 模块：完成取指令阶段的全部工作，包括 pc 的递增、pc 更新、通过 ibus 取指令等。

```
module instr_mem import common::*;(
    input logic clk,
    input logic reset,
    input logic consume,
    input ibus_resp_t ibus_resp,
    input logic [63:0] pcinit,

    output logic fetch_ok,
    output logic [31:0] instr,
    output ibus_req_t ibus_req,
    output logic [63:0] instr_pc
);
```

具体功能：reset 时全部清零。

每一拍，当 consume=1，表示目前的指令被下游消费，fetch_ok 归零。

当目前没有挂起请求且没有已取回但是没有消费的指令时，pending=1，发起新的请求，把发起请求的 pc 锁存起来。收到返回前每一拍保持。

当 ibus_resp.addr_ok 和 data_ok 都是 1，instr 和 pc 传给后面，fetch_ok 写入 1，pc 更新+4，pending 归 0。

- (2) if_id_reg：取指阶段到译码阶段的流水寄存器。

```
module if_id_reg (
    input logic [31:0] instr_f,
    input logic clk,
    input logic reset,
    input logic fetch_ok,
    input logic [63:0] pc_f,

    output logic valid_d,
    output logic [63:0] pc_d,
    output logic [31:0] instr_d
);
```

具体功能：如果 reset 为 1，那么全部归 0。

每一拍，当 fetch_ok 为 1 时，说明取到有效指令，将 instr_f 更新写到 instr_d.pc_f 写到 pc_d，1 写到 valid_d

如果 fetch_ok 为 0，valid 为 0。

- (3) reg_file：存储寄存器的值，组合可读，时序可写。（左图省略了后续的 18 个 test 寄存器）

```
module reg_file(
    input logic [4:0] rs1,
    input logic [4:0] rs2,
    input logic [63:0] writedata,
    input logic [4:0] rd,
    input logic regwrite,
    input logic clk,
    input logic reset,

    output logic [63:0] rs1_val_d,
    output logic [63:0] rs2_val_d,

    output logic [63:0] test_reg_x0,
    output logic [63:0] test_reg_x1,
    output logic [63:0] test_reg_x2,
    output logic [63:0] test_reg_x3,
    output logic [63:0] test_reg_x4,
    output logic [63:0] test_reg_x5,
    output logic [63:0] test_reg_x6,
    output logic [63:0] test_reg_x7,
    output logic [63:0] test_reg_x8,
    output logic [63:0] test_reg_x9,
    output logic [63:0] test_reg_x10,
    output logic [63:0] test_reg_x11,
    output logic [63:0] test_reg_x12,
    output logic [63:0] test_reg_x13,
);
```

具体功能：保存 32 个寄存器的值，[63:0] reg_x0 一直到[63:0] reg_x31

reset 为高电平的时候全部清零。

组合地将 rs1 对应编号的值输出写到 rs1_val_d，将 rs2 对应编号的值输出写到 rs2_val_d

每一个时钟上升沿，当 regwrite 为 1 时，将 writedata 写入 rd 对应编号的寄存器。但是当 rd 为 0 时，不写入。

组合地将 32 个寄存器的值对应写入 test_reg。

(4) control_unit: 通过指令的特殊段来生成控制信号。

接口:

```
module control_unit(  
    input  logic [2:0] funct3,  
    input  logic [6:0] funct7,  
    input  logic [6:0] opcode,  
    output logic      alusign_d,  
    output logic      memwrite_d,  
    output logic [2:0] aluctrl_d,  
    output logic      alusrc_d,  
    output logic      regwrite_d  
);
```

具体功能: 将指令编码翻译成控制信号。控制信号默认为 0。

当 opcode 为 0010011 时, 将 alusrc_d 写为 1, memwrite_d 写为 0, alusign_d 写为 0, regwrite_d 写为 1

在此基础上, 当 funct3 为 000 时, 将 aluctrl_d 写为 0。

当 funct3 为 100 时, 将 aluctrl_d 写为 1。

当 funct3 为 110 时, 将 aluctrl_d 写为 2。

当 funct3 为 111 时, 将 aluctrl_d 写为 3。

当 opcode 为 0110011 时, 将 alusrc_d 写为 0, memwrite_d 写为 0, alusign_d 写为 0, regwrite_d 写为 1

在此基础上, 当 funct3 为 000, 且 funct7 为 0000000 时, 将 aluctrl_d 写为 0。

当 funct3 为 000, 且 funct7 为 0100000 时, 将 aluctrl_d 写为 4。

当 funct3 为 100 时, 将 aluctrl_d 写为 1。

当 funct3 为 110 时, 将 aluctrl_d 写为 2。

当 funct3 为 111 时, 将 aluctrl_d 写为 3。

当 opcode 为 0011011 时, 将 alusrc_d 写为 1, memwrite_d 写为 0, alusign_d 写为 1, regwrite_d 写为 1

在此基础上, 当 funct3 为 000 时, 将 aluctrl_d 写为 0。

当 opcode 为 0111011 时, 将 alusrc_d 写为 0, memwrite_d 写为 0, alusign_d 写为 1, regwrite_d 写为 1

在此基础上，当 funct3 为 000，且 funct7 为 0000000 时，将 aluctrl_d 写为 0。

当 funct3 为 000，且 funct7 为 0100000 时，将 aluctrl_d 写为 4。

(5) sign12to64: 完成 i-type 指令立即数的符号扩展。

```
module sign12to64(  
    input logic [11:0] short_imm,  
    output logic [63:0] long_imm  
);
```

功能：将 12 位 short_imm 符号扩展到 64 位 long_imm

(6) id_exe_reg: 译码阶段到执行阶段的流水寄存器

```
module id_exe_reg(  
    input logic clk,  
    input logic reset,  
    input logic [63:0] rs1_val_d,  
    input logic [63:0] rs2_val_d,  
    input logic [63:0] imm_d,  
    input logic [4:0] rd_d,  
    input logic alusign_d,  
    input logic [2:0] aluctrl_d,  
    input logic alusrc_d,  
    input logic memwrite_d,  
    input logic regwrite_d,  
    input logic [63:0] pc_d,  
    input logic [31:0] instr_d,  
    input logic valid_d,  
  
    output logic valid_e,  
    output logic [31:0] instr_e,  
    output logic [63:0] pc_e,  
    output logic [63:0] rs1_val_e,  
    output logic [63:0] rs2_val_e,  
    output logic [63:0] imm_e,  
    output logic [4:0] rd_e,  
    output logic alusign_e,  
    output logic [2:0] aluctrl_e,  
    output logic alusrc_e,  
    output logic memwrite_e,  
    output logic regwrite_e  
);
```

功能：流水寄存器，
reset 时全部归零。
每一拍将输入的“d”类量写入对应的“e”类量

(7) srcb_mux: 决定第二个算数来自于 reg 还是 imm 的复用器。(itype or rtype)

```
module srcb_mux(  
    input logic [63:0] imm_e,  
    input logic [63:0] rs2_val_e,  
    input logic alusrc_e,  
    output logic [63:0] srcb_e  
);
```

具体功能：复用器。
当 alusrc_e=0 时，将 rs2_val_e 写到 srcb_e
当 alusrc_e=1 时，将 imm_e 写到 srcb_e

(8) alu: 执行运算

```
module alu(  
    input logic [63:0] srca_e,  
    input logic [63:0] srcb_e,  
    input logic [2:0] aluctrl_e,  
    output logic [63:0] alu_result_e  
);
```

具体功能：对 srca_e, srcb_e 根据 aluctrl_e 进行 4 种不同运算。
当 aluctrl_e=0, 输出 srca_e+srcb_e
当 aluctrl_e=1, 输出 srca_e 按位异或 srcb_e

当 aluctrl_e=2, 输出 srca_e 按位或 srcb_e

当 aluctrl_e=3, 输出 srca_e 按位与 srcb_e

当 aluctrl_e=4, 输出 srca_e-srcb_e

忽略溢出, 输出给 alu_result_e

(9) sign32to64: w 指令算术扩展

//功能: 截取64位short_imm的低32位, 符号扩展到64位long_imm

```
module sign32to64(  
    input logic [63:0] short_imm,  
    output logic [63:0] long_imm  
);
```

(10) alures_mux:是否选择 w 指令扩展结果的复用器

//功能: 复用器。

// 当alusign_e=0时, 将alu_result_e写入final_alu_result_e

// 当alusign_e=1时, 将long_alu_result_e写入final_alu_result_e

```
module alures_mux(  
    input logic      alusign_e,  
    input logic [63:0] alu_result_e,  
    input logic [63:0] long_alu_result_e,  
    output logic [63:0] final_alu_result_e  
);
```

(11) ex_mem_reg: 从执行阶段到 mem 阶段的流水寄存器

//功能: 流水寄存器, reset清零。每一拍将输入的“e”类量写入对应的“m”类量

```
module ex_mem_reg(  
    input logic      memwrite_e,  
    input logic [63:0] final_alu_result_e,  
    input logic [4:0] rd_e,  
    input logic      clk,  
    input logic      reset,  
    input logic      regwrite_e,  
    input logic [63:0] pc_e,  
    input logic [31:0] instr_e,  
    input logic      valid_e,  
  
    output logic      valid_m,  
    output logic [63:0] pc_m,  
    output logic [31:0] instr_m,  
    output logic      memwrite_m,  
    output logic [63:0] final_alu_result_m,  
    output logic [4:0] rd_m,  
    output logic      regwrite_m  
);
```

//功能: 流水寄存器, reset清零。每一拍将输入的“m”类量写入对应的“w”类量。

```
module mem_wb_reg (  
    input logic [63:0] aluout_m,  
    input logic [4:0] rd_m,  
    input logic      regwrite_m,  
    input logic      clk,  
    input logic      reset,  
    input logic [63:0] pc_m,  
    input logic [31:0] instr_m,  
    input logic      valid_m,  
  
    output logic [63:0] pc_w,  
    output logic [31:0] instr_w,  
    output logic [63:0] aluout_w,  
    output logic [4:0] rd_w,  
    output logic      regwrite_w,  
    output logic      valid_w  
);
```

(12) mem_wb_reg: mem 阶段到写回阶段的流水寄存器 (见上)

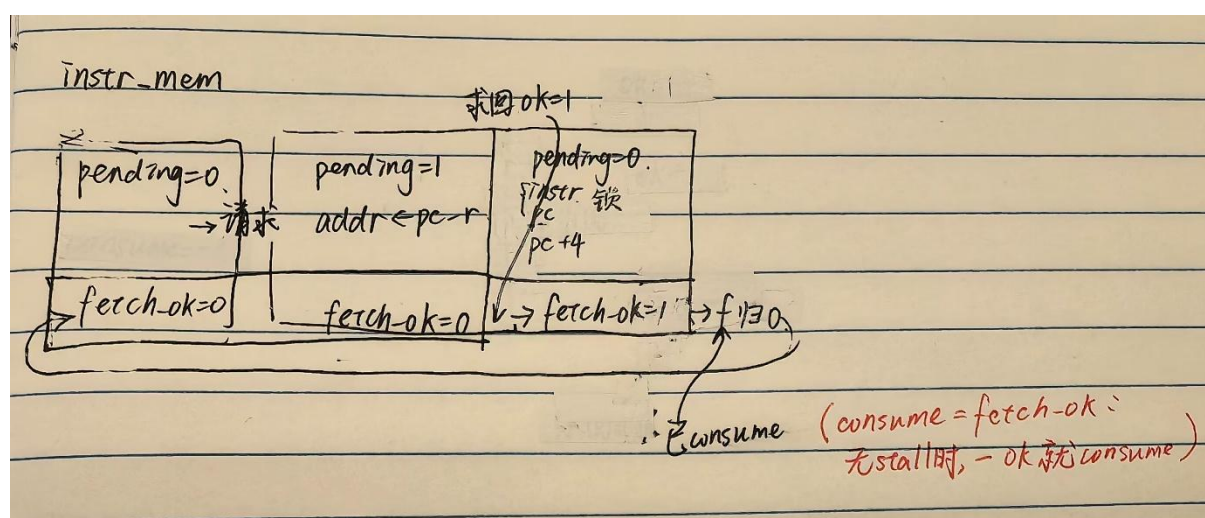
3、datapath 模块思路说明：

Datapath 连接起所有数据通路，即参照图纸上的所有连线，进行了各个量串连。
Datapath 模块调用所有 12 个功能模块，按照图纸进行接口的填写。同时向外暴露 test 所需要的 32 个寄存器的测试值、当前执行完的指令、pc 这几个信号。

三、 调试说明

调试过程中，除了一开始解决的一些 system Verilog 语法问题之外，主要碰到了两个设计的问题。一个出现在取指令阶段，一个出现在 test 接口的填写对应上。

我一开始把 IF 拆成 pcreg、add_adder 和 instr_mem 三个功能模块分开完成，pcreg 负责 pc 更新，addadder 来+4，imem 接到更新 pc 后取指令。imem 需要向 pc_reg 回传一个是否取回指令的信号 pc_en。我一开始的实现是取回的时刻 pc_en 写成 1，下一拍就跳回 0（因为 if_id_reg 一定能在一拍取走指令）。但是由于 pcreg 也是在每拍更新，所以导致每次 pcreg 想要更新的时候，pc_en 都会跳回 0，所以 pcreg 一直停留在第一个 pcinit。为了解决这个问题，把三个功能模块合起来写。逻辑如下图。



第二个问题出现在第四拍左右的地方，也就是第一个指令该写回的时候 diffset 对比不一致。问题出在寄存器的值在 wb 的那一刻就传回了 diffset，但是测试的应该是下一拍。所以这里在 datapath 中增加了一些 test_reg，wb 之后再打一拍变为测试值。

四、 LLM 使用说明

我在整个实验的完成过程与 LLM 的合作方式如下：

在收到实验要求后，我要求 LLM 检索了可以使我更加理解流水线 cpu 的文档、仓库和网页，接下来我用类似于检索增强生成的思想请 LLM 结合这些文档对我进行讲解。这里主要参考了 RSIC-v 指令集的官方文档、《Computer organization and design: The hardware/software

interface (RISC-V edition)》这本书的第四章和一部分中科大的流水线 cpu 的教学资料。

搞清楚大致的设计思路后，我结合老师课堂上使用的 mips 的 cpu 图纸绘制了 resic-v 的图纸，过程中和 LLM 讨论了一些困惑。绘制图纸后，我为每个功能模块书写了详细的注释头，包括模块名称、具体接口和自然语言描述的具体功能，并由 LLM 来写 system Verilog 代码。因为 datapath 的逻辑特别复杂，不是很好描述，所以这部分由我手动来写。最后调试阶段，我和 LLM 讨论了报错信息，这里比较大量的参考了它的建议。

具体会话见下一页附录。

附录：lab 对 IIm 的提问记录（调试过程丢失了一部分）

- 什么是五级流水线 CPU 架构？它和单周期 CPU 的区别是什么？
- RISC-V 指令集，它是 32 位的还是 64 位的？
- 如果我现在想了解 64 位运算中的算术运算和逻辑运算指令，我应该查哪个文档？
- 第四章有 addiw、subw 和 addw，但是我还要找 addi、xori、add、sub 等，这里没有，为什么？
- 因为我最终想实现的是 64 位，这两个（RV32I 和 RV64I）在指令上一样吗？还是说我需要找到文档中说明二者区别的部分？
- RISC-V 指令集的寄存器有固定的数量、名称和功用吗？
- 我一直以为 RISC-V 指令集只包括指令部分，原来还有寄存器规范。那么除了这些还有别的吗？
- 一个 CPU 内核就是一个指令集的实现，而指令集是 CPU 内核的说明书或者规划书”，这样理解对吗？
- ISA、微架构、RTL、电路实现四者的关系是什么？
- 这个 lab 是不是就是在 RISC-V 这个 ISA 的基础上，遵循 5 级 pipeline 的微架构，进行 RTL 设计，实现 CPU 内核？这样说有什么问题？
- 请你阅读这个 RISC-V 手册，后面按这个手册回答我的问题。
- 请梳理手册中关于 addi, xori, ori, andi, add, sub, and, or, xor, addiw, addw, subw 这十二条指令的内容，完整提供给我。
- 符号扩展是什么意思？是不是全部补零？具体编码是什么样的？
- 为什么 subw 也有“溢出忽略”？不是已经符号扩展了吗？
- 每一个 register 的对应编码是什么？
- 请你介绍 wiki 实验准备里“内存总线”部分，尤其是代码，结合访存讲解其中提供的 fetch 代码。
- 整个 core 设计到底需要写什么？是否应该包括译码器、controller 之类的 RTL 模块？
- 是不是还需要写五级流水线分别的内容？是不是这五个阶段分别调用写好的 RTL 模块？
- 除此之外是不是还要写 datapath？

- 这些 RTL 是课程已经把微架构设计好了，还是需要我自己设计？
- 如果需要我自己设计，我能不能参考一些网上的 RISC-V RTL 结构？
- 能不能给我一些中英文网站、可靠的相关设计文档和网页？
- 对于“既不想完全靠 LLM，又不想自己设计得脱离主流”的担忧，你有什么建议？
- 中科大课件打不开是怎么回事？要不你直接把课件内容提供给我？
- COD RISC-V Edition 的 datapath / pipelining 章节，我应该看哪几个章节？具体编号是什么？
- 为什么 ALU 的输出除了 ALU 计算结果之外，还有一个 Zero？
- 明明说 regfile 需要时钟控制，但是输入和控制信号都没有 clk，为什么？
- 如实翻译并讲解一下这些页面。
- 为什么需要数据存储器单元？立即数不是指令里自带的吗？
- 在我的 lab 中不需要这个 data memory，是这样吗？
- 这一章节的 datapath，可以理解成单周期吗？
- 这一章好像还没有涉及 I-type 的指令？
- 请继续按这种方式讲解 4.4 A Simple Implementation Scheme。需要我上传图片吗？最好你自己查 textbook 来讲。
- 继续按这种方式讲解 4.5，我想继续了解流水线。
- 你举的 $200\text{ ps} \times 5$ 的例子我没理解，为什么不是 1000 ps ？
- 我现在理解五级流水线的意思了，但是我对于怎么把它落实在代码上完全没有想法。
- 我觉得它和我现在 datapath 和 controller 的逻辑找不到关联，所以我应该看 4.6 吗？4.6 可以解决我的疑惑吗？
- 我还是不理解 800 ps 的例子。既然流水线各个阶段并行，每个阶段时间不是都应该一样、都取最长的吗？为什么不同指令的执行时间不同？
- 请按照我们之前合作的模式讲解 4.6。
- 我计划先自己梳理 12 条指令的 5 个阶段，分别需要哪些 module（状态元件和计算元件、控制信号、阶段之间要存什么），再去看你推荐的 repo。你觉得呢？
- 在整个过程中我会有疑惑需要你解答。现在我想问，译码具体是什么模块来完成的？你可以结合 repo Varunkumar0610/RISC-V-32I-5-stage-Pipeline-Core 回答。

- 立即数的符号扩展是在执行阶段吗？还是在译码阶段？
- ImmGen 有的时候是从 32 位扩展到 64 位，有的时候是 12 位扩展到 64 位，这个怎么区分？
- 下面我给你讲一下我的 addi 的五阶段，你看看我有什么漏洞，参考 repo 里面和我的区别。
- 我这样理解 addi 的五阶段有没有漏洞？和 repo 相比差在哪里？
- 流水寄存器放在哪里呢？应该放在 regfile 里吗，还是重新写一个 regfile？
- 可是 reg 不是只能有 32 个吗？怎么办？
- PC 每次都要 +4，而且 lab 还要求初始时给一个 pcinit。这样的话应该怎么做？
- 我对于 PC module 的设计有点懵。
- 我现在感觉编译好复杂啊，要控制很少的寄存器完成很多事情
- 讲一下这个图从译码阶段传到 exe 阶段的路和 rde 分别是什么意思？我只能理解 rd。另外这个 regdst 应该的意思是写回的 reg 吧，为什么有一个 mux 呢？他在选择什么
- 那么 rsicv 里面，没有这个步骤吧，就传 rd 就行了？我现在在对照这个图画我的 rsicv 版本。
- 在刚刚 mips 图的基础上，如果用 riscv 的规则来套（大部分逻辑应该是一样的），对于 addiw 这样的指令，如果要最后要取 rd 的低 32 位符号扩展，那是不是还得在 wb 阶段加一个符号扩展原件？
- riscv 的地址是多少维度
- 下面我来提供模块的接口、名称和实现方法，你来写 system verilog 代码
- 我们一个一个写。模块名称：alu_adder; 接口 ... 功能：输出 pc+4
- sv 语言里面注释是//吗
- 时钟信号是多少位的
- 继续写 system verilog 代码。//模块名称：if_id_reg ... 功能：每一拍将 instrF 更新写到 instrD
- 写 sv 代码。//模块名称：sign12to64 ... 功能：将 12 位 short_imm 符号扩展到 64 位 long_imm
- 写 sv 代码。//模块名称：id_ex_reg ... 功能：流水寄存器，每一拍将输入的“d”类量写入对应的“e”类量

- 减法在 alu 中实现吗
- //模块名称: alu ... 功能: 根据 aluctrl_e 进行 5 种运算
- 写 sv 代码 //模块名称: srcb_mux ...
- 确实错了, 按照现在的写。//模块名称: srcb_mux ...
- //模块名称: sign32to64 ... 写 sv 代码
- 写 sv 代码。//模块名称: alures_mux ...
- 写 sv 代码。//模块名称: ex_mem_reg ...
- 写 sv 代码。//模块名称: mem_wb_reg ...
- 写 sv 代码。//模块名称: control_unit ...
- 我现在已经完成了大部分代码, 剩余有 pc_reg, instr_mem 和 reg_file 以及最终的 datapath 不太会写。我希望你结合课程 wiki 和下面给你提供的 core 代码, 指导我 (不要告诉我答案, 可以进行方向上的点拨)
- 我的课程 wiki: 你看看能不能访问。结合这两个一起说 <https://github.com/26-Arch/26-Arch/wiki/Lab-1>
- pc 这里, 是不是需要一个信号, 表示现在是最开始
- 这么设计可以吗? 先不考虑阻塞。//模块名称: pc_reg ... 功能: 当 reset 为 1 时, 将 pcinit 的值写入 pc_next; 当 reset 为 0 时, 将 pc+4 的值写入 pc_next
- 是的我糊涂了。。现在这样呢? //模块名称: pc_reg ... 功能: 当 reset 为 1 时, 将 pcinit 的值写入 pc_next; 当 reset 为 0 时, 将 pc 的值写入 pc_next
- 这样对吗。//模块名称: pc_reg ... 接口里加了 clk, 输出 pc_next
- 现在呢 //模块名称: pc_reg ... 输入 pc_next, 输出 pc
- 写 sv 代码。//模块名称: pc_reg ...
- 我现在要写 regfile。这个写使能、时钟和 wb 阶段时序写, 具体应该怎么实现呢?
- 我知道了, 那写使能怎么给呢
- 所以我在 controlunit 这个线路补一个写使能?
- 是不是需要 reset, 在最开始的时候初始化呢? 我不太明白, 你写一个简单的例子让我看看
- 我问的是 regfile, 只回答这个就可以

- 我是不是需要向外提供寄存器接口？因为好像 core 的测试里面需要测试 reg
- 方式 1，我的接口这么写可以吗 //接口：... output logic [31:0] reg[63:0]
- //模块名称：reg_file ...（给出一长串接口）
- 所以说 32 个 reg 的值其实是保存在这个模块内部的？我以为要在外部
- //模块名称：reg_file ...（完整功能描述）这样对吗
- 写 sv 代码。//模块名称：reg_file ...
- 我在想这个 regfile，应该需要在 reset 的时候归零吧？我觉得现在没有实现这个
- 给我代码。
- 一条 instr 是 32 位吗
- 看一下这里的取指令逻辑。我想问一下这个 fetchok 是必须有的吗？我感觉我用流水寄存器并不需要啊，我不理解。z 这是这部分的课程 Wiki，你结合来讲
<https://github.com/26-Arch/26-Arch/wiki/实验讲解>
- 我的 instrmem 这么设计，可以吗？//模块名称：instr_mem //接口：input logic [63:0] pc // output logic [31:0] instr
- 我是 b，应该怎么写接口呢？功能大概什么样呢
- 输出结构体。你直接给我写一个 imem 让我看看。
- 这个 common 是什么意思？我不太理解这个 sv 语法。我给你看助教给的 common。
（后面贴了完整 common.sv）
- 写吧，让我参考一下
- 口述一下这个模块的大概功能
- 我不能修改 common.sv
- 我在想这个 regfile，应该需要在 reset 的时候归零吧？我觉得现在没有实现这个
- 给我代码。
- 这个 fetch_en 是干什么的？一定要有吗？
- 看一下这个怎么样。//模块名称：instr_mem ... 功能：将 pc 写入 instr_pc ...
- 现在呢？//模块名称：instr_mem ...
- 要不你按照我的格式写一个头，让我看看？
- 你先写写

- 我给你看我的报错信息。另外我还可以提供课程 lab 的其他信息（例如总线设置和指令要求等），如果你需要可以提出。